

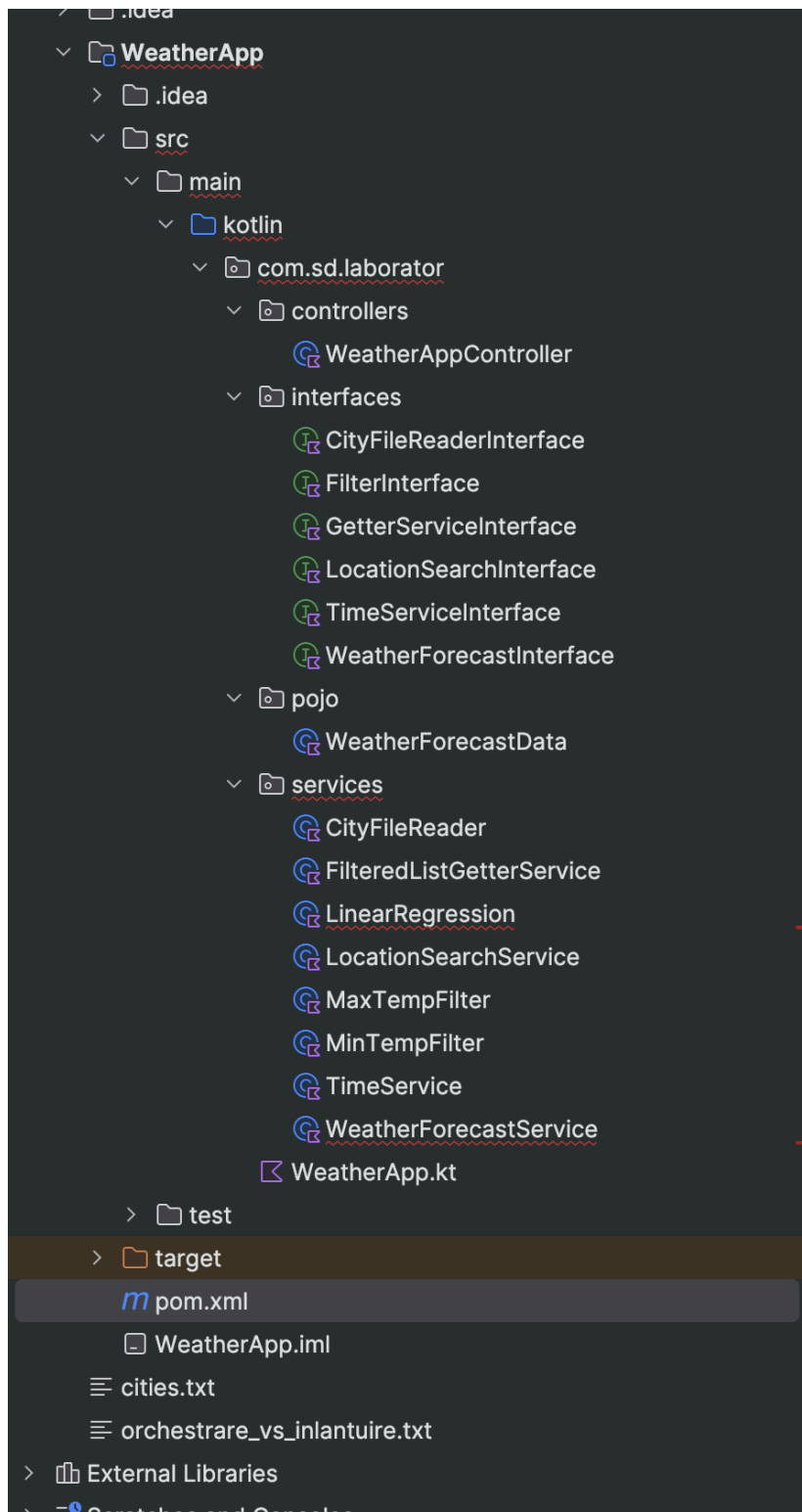
Pornind de la exemplul de proiectare si implementare a unei aplicatii care primeste informatii meteo (adică utilizând si serviciile de acolo eventual cu modificări) să se realizeze următoarele:

- un serviciu care citeste dintr-un fisier (in ce format dorim) o listă de orase;
- să se aplice modelul de proiectare de tip filtru în proiectarea si implementarea unui serviciu care extrage din fluxul de date informatiile pe baza numelui orasului (utilizând serviciul anterior)
- să se proiecteze si să se implementeze un serviciu care utilizând serviciul de filtrare va aplica o regresie liniară peste datele rezultate si apoi se va afisa unghiul dreptei rezultate.

Daca se considera necesar se mai pot crea oricâte servicii se doreste conform gândirii studentului.

Se va utiliza orchestrarea si respectarea principilor SOLID pentru microservicii. La prezentare se va arata diagrama de clase si se va explica aceasta impreuna cu gradul de respectare a principilor. Apoi va urma discutia pe cod.

IMPLEMENTAREA FARA REGRESIE LINIARA



NOTITE

1. In filteredListGetterService modifica path-ul la "fileToRead"
2. Creezi fisierul cities.txt
3. Sa stii diferenta intre orchestrare si Inlantuire

WeatherAppController

```
package com.sd.laborator.services

import com.sd.laborator.interfaces.WeatherForecastInterface
import com.sd.laborator.pojo.WeatherForecastData
import org.json.JSONObject
import org.springframework.stereotype.Service
import java.net.URL
import kotlin.math.roundToInt

@Service
class WeatherForecastService(
    private val timeService: TimeService,
) : WeatherForecastInterface {
    override fun getForecastData(locationId: Int): WeatherForecastData {
        // ID-ul locației nu trebuie codificat, deoarece este numeric
        val forecastDataURL =
            URL("https://www.metaweather.com/api/location/$locationId/")

        // preluare conținut răspuns HTTP la o cerere GET către URL-ul de mai
        // sus
        val rawResponse: String = forecastDataURL.readText()

        // parsare obiect JSON primit
        val responseRootObject = JSONObject(rawResponse)
        val weatherDataObject =
            responseRootObject.getJSONArray("consolidated_weather").getJSONObject(0)

        // construire și returnare obiect POJO care încapsulează datele meteo
        return WeatherForecastData(
            location = responseRootObject.getString("title"),
            date = timeService.getCurrentTime(),
            weatherState = weatherDataObject.getString("weather_state_name"),
            weatherStateIconURL =
                "https://www.metaweather.com/static/img/weather/png/${weatherDataObject.getString("weather_state_abbr")}.png",
            windDirection =
                weatherDataObject.getString("wind_direction_compass"),
            windSpeed = weatherDataObject.getFloat("wind_speed").roundToInt(),
            minTemp = weatherDataObject.getFloat("min_temp").roundToInt(),
            maxTemp = weatherDataObject.getFloat("max_temp").roundToInt(),
            currentTemp = weatherDataObject.getFloat("the_temp").roundToInt(),
            humidity = weatherDataObject.getFloat("humidity").roundToInt(),
        )
    }
}
```

```
}
```

CityFileReaderInterface

```
package com.sd.laborator.interfaces

import java.io.File

interface CityFileReaderInterface {
    fun read(file: File): MutableList<String>
}
```

FilterInterface

```
package com.sd.laborator.interfaces

import com.sd.laborator.pojo.WeatherForecastData

interface FilterInterface {
    fun execute(data: WeatherForecastData): Boolean
}
```

GetterServiceInterface

```
package com.sd.laborator.interfaces

interface GetterServiceInterface {
    fun get(): String
}
```

LocationSearchInterface

```
package com.sd.laborator.interfaces

interface LocationSearchInterface {
    fun getLocationId(locationName: String): Int
}
```

TimeServiceInterface

```
package com.sd.laborator.interfaces
```

```
interface TimeServiceInterface {  
    fun getCurrentTime(): String  
}
```

WeatherForecastInterface

```
package com.sd.laborator.interfaces  
  
import com.sd.laborator.pojo.WeatherForecastData  
  
interface WeatherForecastInterface {  
    fun getForecastData(locationId: Int): WeatherForecastData  
}
```

WeatherForecastData

```
package com.sd.laborator.pojo  
  
data class WeatherForecastData(  
    var location: String,  
    var date: String,  
    var weatherState: String,  
    var weatherStateIconURL: String,  
    var windDirection: String,  
    var windSpeed: Int, // km/h  
    var minTemp: Int, // grade celsius  
    var maxTemp: Int,  
    var currentTemp: Int,  
    var humidity: Int,  
)
```

CityFileReader

```
package com.sd.laborator.services  
  
import com.sd.laborator.interfaces.CityFileReaderInterface  
import java.io.File  
  
class CityFileReader : CityFileReaderInterface {  
    var cities: MutableList<String> = mutableListOf()  
  
    override fun read(file: File): MutableList<String> {  
        file.forEachLine { cities.add(it) }  
    }  
}
```

```

        return cities
    }
}

```

FilteredListGetterService

```
package com.sd.laborator.services

import com.sd.laborator.interfaces.FilterInterface
import com.sd.laborator.interfaces.GetterServiceInterface
import com.sd.laborator.pojo.WeatherForecastData
import java.io.File

class FilteredListGetterService(
    var locationSearchService: LocationSearchService,
    var weatherForecastService: WeatherForecastService,
) : GetterServiceInterface {
    val fileToRead =
File("/Users/andreighiurca/Downloads/SD-EXAMEN/5/cities.txt")
    val citiesList = CityFileReader().read(fileToRead)

    val filteredCitiesList: MutableList<WeatherForecastData> = mutableListOf()
    val filtersList: List<FilterInterface> = listOf() // DE ADAUGAT FILTRE
    @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

    override fun get(): String {
        citiesList.forEach { location ->
            val locationId = locationSearchService.getLocationId(location)

            // dacă locația nu a fost găsită, răspunsul va fi corespunzător
            if (locationId == -1) {
                return "Nu s-au putut gasi date meteo pentru cuvintele cheie
\n$location\n!"
            }

            val rawForecastData: WeatherForecastData =
weatherForecastService.getForecastData(locationId)

            // se verifica fiecare filtru sa fie indeplinit
            var ok = 1
            filtersList.forEach { filter ->
                if (!filter.execute(rawForecastData)) {
                    ok = 0
                }
            }

            if (ok == 1) {
                filteredCitiesList.add(rawForecastData)
            }
        }
    }
}
```

```

    }
}

return filteredCitiesList.toString()
}
}

```

LinearRegression

```

package com.sd.laborator.services

class LinearRegression(
    var getterService: FilteredListGetterService,
) {
    fun getSlope(): Double {
        val n = getterService.filteredCitiesList.size
        var sumXY: Double = 0.0
        var sumX: Double = 0.0
        var sumY: Double = 0.0
        var sumXsquare: Double = 0.0

        getterService.filteredCitiesList.forEach {
            sumXY += it.minTemp * it.maxTemp
            sumX += it.minTemp
            sumY += it.maxTemp
            sumXsquare += it.minTemp * it.minTemp
        }

        return ((n * sumXY) - (sumX * sumY)) / ((n * sumXsquare) - sumXsquare)
    }
}

```

LocationSearchService

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.LocationSearchInterface
import org.springframework.stereotype.Service
import java.net.URL
import org.json.JSONObject
import java.net.URLEncoder
import java.nio.charset.StandardCharsets

@Service
class LocationSearchService : LocationSearchInterface {
    override fun getLocationId(locationName: String): Int {

```

```

        // codificare parametru URL (deoarece poate conține caractere speciale)
        val encodedLocationName = URLEncoder.encode(locationName,
StandardCharsets.UTF_8.toString())

        // construire obiect de tip URL
        val locationSearchURL =
URL("https://www.metaweather.com/api/location/search/?query=$encodedLocationNam
e")

        // preluare raspuns HTTP (se face cerere GET și se preia conținutul
răspunsului sub formă de text)
        val rawResponse: String = locationSearchURL.readText()

        // parsare obiect JSON
        val responseObject = JSONObject("{\"data\": ${rawResponse}}")
        val responseContentObject =
responseObject.getJSONArray("data").takeUnless { it.isEmpty }
                ?.getJSONObject(0)
        return responseContentObject?.getInt("woeid") ?: -1
    }
}

```

MaxTempFilter

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.FilterInterface
import com.sd.laborator.pojo.WeatherForecastData

class MaxTempFilter: FilterInterface {
    override fun execute(data: WeatherForecastData): Boolean {
        if(data.maxTemp<25)
            return true
        return false
    }
}

```

MinTempFilter

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.FilterInterface
import com.sd.laborator.pojo.WeatherForecastData

class MinTempFilter: FilterInterface {

```

```

        override fun execute(data: WeatherForecastData): Boolean {
            if(data.minTemp>10)
                return true
            return false
        }
    }
}

```

TimeService

```

package com.sd.laborator.services
import com.sd.laborator.interfaces.TimeServiceInterface
import org.springframework.stereotype.Service
import java.text.SimpleDateFormat
import java.util.*

@Service
class TimeService : TimeServiceInterface {
    override fun getCurrentTime():String {
        val formatter = SimpleDateFormat("dd/MM/yyyy HH:mm:ss")
        return formatter.format(Date())
    }
}

```

WeatherForecastService

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.WeatherForecastInterface
import com.sd.laborator.pojo.WeatherForecastData
import org.json.JSONObject
import org.springframework.stereotype.Service
import java.net.URL
import kotlin.math.roundToInt

@Service
class WeatherForecastService(
    private val timeService: TimeService,
) : WeatherForecastInterface {
    override fun getForecastData(locationId: Int): WeatherForecastData {
        // ID-ul locației nu trebuie codificat, deoarece este numeric
        val forecastDataURL =
URL("https://www.metaweather.com/api/location/$locationId/")

        // preluare conținut răspuns HTTP la o cerere GET către URL-ul de mai
sus
    }
}

```

```

        val rawResponse: String = forecastDataURL.readText()

        // parsare obiect JSON primit
        val responseRootObject = JSONObject(rawResponse)
        val weatherDataObject =
responseRootObject.getJSONArray("consolidated_weather").getJSONObject(0)

        // construire și returnare obiect POJO care încapsulează datele meteo
        return WeatherForecastData(
            location = responseRootObject.getString("title"),
            date = timeService.getCurrentTime(),
            weatherState = weatherDataObject.getString("weather_state_name"),
            weatherStateIconURL =
"https://www.metaweather.com/static/img/weather/png/${weatherDataObject.getStri
ng("weather_state_abbr")}.png",
            windDirection =
weatherDataObject.getString("wind_direction_compass"),
            windSpeed = weatherDataObject.getFloat("wind_speed").roundToInt(),
            minTemp = weatherDataObject.getFloat("min_temp").roundToInt(),
            maxTemp = weatherDataObject.getFloat("max_temp").roundToInt(),
            currentTemp = weatherDataObject.getFloat("the_temp").roundToInt(),
            humidity = weatherDataObject.getFloat("humidity").roundToInt(),
        )
    }
}

```

[WeatherApp.kt](#)

```

package com.sd.laborator

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
open class WeatherApp

fun main(args: Array<String>) {
    runApplication<WeatherApp>()
}

```

Pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.sd.laborator</groupId>
    <artifactId>WeatherApp</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <name>com.sd.laborator WeatherApp</name>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.1.9.RELEASE</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <kotlin.version>1.3.61</kotlin.version>
        <kotlin.code.style>official</kotlin.code.style>
        <junit.version>4.12</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-stdlib</artifactId>
            <version>${kotlin.version}</version>
        </dependency>
        <dependency>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-test-junit</artifactId>
            <version>${kotlin.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
```

```

        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-reflect</artifactId>
        <version>${kotlin.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <version>${project.parent.version}</version>
    </dependency>
    <dependency>
        <groupId>org.json</groupId>
        <artifactId>json</artifactId>
        <version>20190722</version>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <compilerPlugins>
                            <plugin>spring</plugin>
                        </compilerPlugins>
                    </configuration>
                </execution>
                <execution>
                    <id>test-compile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
            </executions>
            <dependencies>
                <dependency>
                    <groupId>org.jetbrains.kotlin</groupId>

```

```
        <artifactId>kotlin-maven-allopen</artifactId>
        <version>${kotlin.version}</version>
    </dependency>
</dependencies>
</plugin>

<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```